

3IRC

Conception et Programmation Orientées Objet

Ateliers



Objectifs du Module

Compétences visées	Moyen	Pédagogie active	Evaluation
- Être de plus en plus capable de concevoir et développer des applications : Fiables et robustes Faciles à développer Faciles à maintenir (corrections et évolutions mineures) Faciles à étendre (ajout nouvelles fonctionnalités) Dont on peut facilement réutiliser des composants. - Pour atteindre ces objectifs : - Il convient de mettre en œuvre les bonnes pratiques de conception et de programmation objet en respectant les principes S.O.L.I.D. - En s'appuyant sur les piliers de la POO que sont : - L'abstraction. - L'encapsulation. - L'héritage. - Le polymorphisme.	- Un atelier de démarrage pour les débutants (autre document). - Un projet fil rouge qu'il s'agira d'enrichir au fur et à mesure des ateliers à partir, dans un premier temps, de spécifications techniques précises puis de spécifications fonctionnelles détaillées. - L'architecture et le squelette sont définis, un certain nombre de interfaces/classes sont déjà fournies complètes ou à compléter. - Ce projet est un simulateur ultra simplifié de jeu de carte en ligne. Le premier jeu développé est un jeu de bataille qui sera implémenté avec 2 méthodes différentes de prise (en cas de bataille). - La conception est telle que les évolutions du Model sont possibles pour jouer à d'autres jeux (belote, etc.) avec plusieurs IHM (2 Views sont fournies) et différents Controller.	- Des vidéos à visionner en amont des séances pour acquérir les connaissances. - Des recherches à effectuer en équipe avec transfert de compétences. - Des développements à effectuer seul, en binôme ou en équipe. - Un Carnet de Bord à compléter au fur et à mesure pour capitaliser connaissances et compétences. - Des enseignants facilitateurs qui vérifient la bonne acquisition des compétences et qui fournissent des compléments d'information. 	DS papier/crayon : évalue la capacité à s'approprier la conception et le code d'un programme existant et à le maintenir en Java. Il s'agit d'être capable : - De justifier ou critiquer certains choix de conception objet. - De justifier ou critiquer le choix de certaines structures de données. - De dire ce que font certaines instructions en expliquant leur intérêt pour le programme. - De modifier des portions de codes volontairement mal conçues ou mal écrites. - D'ajouter des fonctionnalités inexistantes. Participation active aux différents ateliers : - Les sujets et corrections étant fournies, ce ne sont pas les résultats qui seront évalués mais la démarche, l'envie d'apprendre, etc.

Spécifications fonctionnelles et techniques

Cahier des charges

Ce projet est un simulateur ultra simplifié de jeu de carte en ligne.

- Il existe plusieurs types de Game (Bataille, Belote, etc.) avec chacun des règles différentes en particulier pour savoir quel joueur remporte un pli, une partie, un tournoi, etc. Dans un 1^{er} temps :
 - Seules 2 versions du jeu de Bataille seront développées avec des méthodes différentes de prise (en cas de bataille). 1 version où le Roi de Pique l'emporte sur le Roi de Cœur, 1 version où il y a bataille et dans ce cas, les joueurs jouent une nouvelle carte et celui qui a la plus haute remporte toutes les cartes du tapis.
 - Les notions de parties et de gains (nb de points) ne sont pas gérées.
- Il existe différents types de cartes définies par une valeur et une couleur (Ex. 7 de Pique). Il n'est pour l'instant pas prévu d'autres types de cartes pour jouer par exemple au Tarot, au Uno, etc. mais la conception de l'application ne devrait pas empêcher cette évolution.
- Chaque joueur joue alternativement et selon les règles du jeu, 1 joueur ou une équipe remporte le pli et à la fin, 1 joueur ou une équipe remporte la partie. Dans un 1^{er} temps il n'est pas prévu de gérer des équipes, mais (etc.).
- L'état du « tapis », le gagnant de chaque pli et le vainqueur de la partie s'affiche sur l'écran des joueurs. Dans un 1^{er} temps, 1 seul écran pour tous les joueurs mais il devra être possible de jouer sur des écrans différents, mais (etc.).

Analyse

L'analyse du programme a permis d'identifier un certain nombre d'objets du monde réel :

- Des Cartes qui ont 1 Valeur (Valet), 1 Couleur (Pique), plusieurs Rangs (le 9 rapporte 9 points à la Bataille, 0 points à la Belote ou 14 points s'il est à l'Atout).
- Des « tas » (Collections) de cartes (le paquet de cartes complet, la pioche, la main d'un joueur, les cartes posées sur le tapis, etc.).
- Des Joueurs qui ont 1 nom, 1 Main, 1 Tas de carte gagnées (qui peuvent rejoindre ou non sa main selon le jeu) et éventuellement 1 Pioche. Leurs cartes peuvent être visibles ou cachées. Ils peuvent jouer l'une des cartes de leur Main ou seulement celle du dessus (et la retourner) selon le type de Game.
- Des types de Game avec des règles d'évaluation des plis et de détermination du vainqueur différentes (Bataille, Belote, etc.).
- 1 ou plusieurs IHM (console, graphique, mobile, etc.) qui permettent de gérer les interactions avec les utilisateurs (les incarnations IRL des Joueurs). A travers l'une des View, le choix du type de Game (Bataille), du type de tas de cartes (32 ou 52 cartes) et des noms des utilisateurs sont effectués.

Conception

La conception a traduit cette analyse. Une architecture MVC (Model View Controller) a été pensée pour séparer les responsabilités de chaque package, package qui peut ainsi être développé/modifié sans (grand) impact sur les autres.

- Interfaces et Classes du Model (« Métier »):
 - Des Cartes modélisés par une interface (ICard) et 2 classes qui l'implémentent Card et CardRender. La classe CardRender permet de créer des représentations des Cartes, manipulables par les IHM, mais seuls les objets Card (manipulés par le model) peuvent être rendues visibles ou invisibles. *[Cette distinction a des vertus essentiellement pédagogiques, car rien ne le justifie réellement...]*.
 - Des Collections de Cartes modélisés par une interface (ICardsCollection) et 3 classes qui l'implémentent (Hand, Deck, Board). Une classe abstraite regroupe tous les attributs et méthodes communs à ces 3 classes (AbstractCardsCollection).
 - Des Joueurs modélisés par une interface (IPlayer) et 2 classes qui l'implémentent Player et PlayerRender (même principe que pour CardRender et Card).
 - Des Game modélisés par une interface (IGame) et plusieurs classes qui l'implémentent (ClassicWarGame et NewWarGame). Une classe abstraite regroupe tous les attributs et méthodes communs à tous les jeux de cartes (AbstractGame) et une autre qui regroupe tout ce qui est commun à tous les jeux de Bataille (AbstractWarGame).
 - Des évaluateurs de pli modélisés par une interface (IGameEvaluator) et plusieurs classes qui l'implémentent (ClassicWarGameEvaluator et NewWarGameEvaluator). Une classe abstraite regroupe tous les attributs et méthodes communs à tous les jeux de cartes (AbstractGameEvaluator).
- Interfaces et Classes de la View (IHM) :
 - Des vues modélisées par une interface (IGameView) et plusieurs classes qui l'implémentent (GameConsoleView et GameSwingView). Une classe particulière propage les messages envoyés par le Controller à toutes les Views. Cela simplifie le code du Controller qui ne communique qu'avec 1 seule View alors qu'il y en a réellement plusieurs.
- Interfaces et Classes du Controller (communication entre Model et View) :
 - Un Controller (GameController) qui gère l'enchaînement des actions et la communication entre les IGame et la ou les IGameView.
- Enum utiles dans tous les packages :
 - GameType, CardsCollectionType, Rank, Suit.

La majorité des interfaces et classes nécessaires à l'exécution du programme vous sont fournies de même que des programmes de test. Certaines méthodes sont complètes (Views et Controller) et beaucoup sont à compléter. Certaines autres seront à identifier et à ajouter en fonction des besoins.

Atelier N°1

Atelier 1 - Objectifs

Compétences acquises à travers l'atelier (être capable de...)	Connaissances acquises à travers l'atelier	Étude préliminaire : vidéos à visionner avant l'atelier
• Lire et Ecrire du code Java. • Maîtriser les différents éléments du langage. • Comprendre en quoi la Programmation Orientée Objet facilite les tests, l'écriture (débuggage) et la résistance au changement par rapport à la programmation procédurale (1ère illustration du Principe de Responsabilité Unique).	• Bases du langage Java (types de variables, structures de contrôle, visibilité/portée des variables/méthodes, fonction main()). • Bases de la Programmation Orientée Objet en Java : interface, classe, objet, référence, constructeur, attribut, méthode. • Piliers de la Programmation Orientée Objet : abstraction, encapsulation invocation de méthode et composition et délégation. • Interfaces Comparable, List et Iterator de Java. • Classes Object (méthodes toString() et equals()) et ArrayList de Java.	1. Vidéo Youtube de Xavier Blanc « Programmation Orientée Objet - Cours 1 » (les 13 premières minutes) 2. Vidéo Youtube de Xavier Blanc « Programmation Orientée Objet - Cours 2 : La classe » (14') 3. Vidéo Youtube de Xavier Blanc « Programmation Orientée Objet - Cours 2 : Exemple » (8') 4. Vidéo Youtube de Xavier Blanc « Programmation Orientée Objet - Cours 3 : Interfaces » (les 6 premières minutes - entendre "client/application cliente" à la place "d'utilisateur".

Atelier 1 - Déroulement

Objectifs et Moyens

Programmer les premières classes « métier » du projet « fil-rouge ».

- Classe Card : une carte peut être visible ou pas et peut être comparée à une autre. Elle a une valeur (Rank) et une couleur (Suit).
- Classe Hand qui gère une List de Card. On peut y ajouter, supprimer, trouver, révéler, cacher une Card,
- Classe Player qui contient 2 objets Hand : sa main et ses plis. On peut demander à un joueur de jouer une carte, de ramasser un pli, savoir s'il a gagné le dernier pli, toutes les cartes du jeu, le jeu, etc.

Un programme permet de tester pas à pas les différentes méthodes des classes.

1^{ère} activité : installation des sources et 1^{er} test

- Créez un projet dans votre IDE à partir des sources du projet Fil-Rouge disponibles sur le e-campus (respectez bien les packages) et
- Exécutez le programme TestAtelier1 dans le package launcher. Normalement, la trace d'exécution (ce qui s'affiche) est complète mais les résultats ne correspondent pas à ceux attendus (Cf. ci-dessous).
- Mettez toutes les instructions de la fonction main() en commentaire et exécutez : plus rien ne s'affiche. Vous dé-commenterez les instructions au fur et à mesure de vos développements.

2^{ème} activité : développement

- Le programme de test est à dé-commenter au fur et à mesure des développements. Le résultat à obtenir est indiqué en commentaire à la fin de chaque ligne de code et en Annexe (fichier « Atelier 1 – Trace »).
- Les méthodes sont à compléter au fur et à mesure dans les classes du package model.
- Testez chaque ligne/bloc de test avant de passer à la suivante.
- Complétez le Carnet de Bord au fur et à mesure.

3^{ème} activité : correction du Carnet de Bord

Atelier 1 - Trucs et Astuces / Ce qu'il faut retenir

Éléments de base du langage, Tests, Boucles

- Principes généraux :

- Toute ligne de code est nécessairement définie dans une classe.
- Les instructions se finissent par un « ; », les blocs d'instructions sont délimités par des « { } »
- Le point d'entrée d'un programme est la fonction main().
- Toutes les variables sont typées. Ce sont soit des « primitive » (elles sont d'un type primitif : int, boolean, etc.), soit ce sont des références vers des tableaux (elles contiennent l'adresse du 1^{er} élément du tableau) ou vers des objets (elles contiennent l'adresse de l'objet).
- Le type d'une variable définit l'espace qu'elle occupe en mémoire et le comportement qu'on en attend : ainsi une variable de type int occupe 4 octets et on peut réaliser un certains nombre d'opérations avec (arithmétiques, logiques, etc.)

- Syntaxe de l'instruction if-else :

```
if (condition == true) {  
    // actions quand condition vraie  
}  
else {  
    // actions quand condition fausse  
}
```

- Syntaxe de l'instruction while :

```
while (condition) {  
    // actions quand condition vraie  
}
```

- Syntaxe de l'instruction do-while :

```
if (condition) {  
    do {  
        // actions quand condition vraie  
    } while (condition) ;  
}
```

- Syntaxe de l'instruction for (cas général) :

```
for (initialisation; condition d'arrêt; increment){
    // actions tant que condition vraie
}
```

- Syntaxe de l'instruction for (cas d'un tableau) :

```
for (initialisation; condition d'arrêt; increment){
    // actions tant que condition vraie sur refVersDebutTableau[i]
}
```

Exemple

```
int refVersDebutTableau [10] = {12, 25, 18} ;
for (int i = 0; i < refVersDebutTableau.length; i++){
    // actions tant que condition vraie sur refVersDebutTableau [i]
}
```

- Syntaxe de l'instruction for-loop (cas d'un tableau) :

```
for (typeElement maRefVersUnElement : refVersDebutTableau){
    // actions tant qu'il existe des éléments
    // maRefVersUnElement référence successivement tous les éléments du tableau
}
```

Règles de nommage

- Noms de variables, méthodes, packages : camelCase
- Noms de classes, interfaces, enum : PascalCase (ou UpperCamelCase)
- Noms de constantes : SCREAMING_SNAKE_CASE
- Les accesseurs et mutateurs des attributs sont par nommés par convention getXXX() et setXXX(), où XXX est le nom de l'attribut. Ex getName() ;

Interface, Classe, Objet, énumération, Référence, Constructeur, Attributs, Méthodes

- Définir une classe veut dire définir un type structuré, i.e. des données à stocker en mémoire (ses attributs) et un comportement (ses méthodes). Exemple, la classe Card définit 3 attributs (couleur, rang, visibilité) et des méthodes (cacher/rendre visible la carte, la comparer à une autre, etc.).
- Une classe permet de créer des objets (on parle d'instance de classe).
 - Les attributs sont les propriétés de l'objet (ses données, ce qu'il sait/connaît).
 - Les méthodes sont les opérations/fonctionnalités dont l'objet est capable (les services qu'il peut rendre, ce qu'il sait faire, etc.).
 - Les méthodes d'un objet ne sont accessibles en Java qu'en passant par une variable qui connaît son adresse. Cette variable s'appelle une référence (pointeur en langage C).
- Un objet est créé lorsque l'on fait appel à l'un de ses constructeurs avec l'instruction new:
 - Exemple Card card = **new** (Card(Rank._ROI, Suit.CARREAU) ;
 - Il peut y avoir plusieurs façons de créer l'objet, donc plusieurs constructeurs.
 - Si le programmeur n'a pas explicitement écrit un constructeur le compilateur en synthétise un sans paramètre (= en crée un dans le byteCode) qui initialise les attributs avec des valeurs par défaut (0 pour les numériques, false pour les booléens et null pour les références). On appelle ce constructeur le constructeur par défaut (défaut ⇔ absence).
 - Attention, card n'est pas un objet : c'est une variable qui référence un objet (autrement dit une référence dont la valeur est l'adresse de l'objet).
- Les applications/méthodes clientes d'un objet (ou dit plus rapidement « les clients ») ont besoin que l'objet sache faire des choses (ses méthodes) mais ne veulent pas savoir comment il les fait ni quel est son type réel. Elles vont donc les déclarer en tant qu'instance d'Interface et non de Classe :
 - Une interface définit un Type sans se soucier des détails d'implémentation comme le fait une classe (ni stockage des données, ni détail des algorithmes). Elle se contente de décrire le comportement attendu des objets (déclaration de méthodes).
 - Exemple : Une interface munie d'une méthode rouler() pourrait être implémentée par une classe Voiture, Trottinette, Ballon, PetitPois, etc. La seule info qui intéresse le client est de savoir que l'objet sait rouler()).
 - La plupart du temps les applications clientes ne connaissent pas les types réels qui sont implémentés (les classes) et délèguent à une fabrique (Factory) le soin de créer les objets.
 - Une classe qui implémente une interface doit implémenter toutes ces méthodes (elle ou ses classes dérivées – Cf. plus tard, notion d'héritage).
- Une énumération (enum) est une classe, dont les valeurs (des constantes) sont les seules instances possibles de cette classe.

Méthode equals() de la classe Object et compareTo() de l'interface Comparable

- L'objectif de la méthode « boolean equals(Object obj) » est de vérifier l'égalité sémantique de 2 objets.
 - Elle est déjà codée dans la classe Object et est automatiquement héritée dans vos propres classes.
 - Mais si vous ne la redéfinissez pas dans vos propres classes, l'instruction « c1.equals(c3) » retournera « false » car même si les objets semblent égaux (leurs attributs ont les mêmes valeurs), la méthode de la classe Object (héritée) se contente de comparer les valeurs des références (c1 == c3 ?).
- Pour comparer 2 objets d'une même classe (plus grand, plus petit, égal), celle-ci doit implémenter l'interface Comparable :
 - et définir la méthode compareTo().
 - compareTo() compare les éléments selon ce que l'on appelle « l'ordre naturel ». [Vous étudierez plus tard d'autres comparateurs]

Classe String et méthode toString()

- Les objets de type String contiennent une chaîne de caractère.
 - Si l'on déclare String maChaine = {« Hello Word »}, la variable maChaine contient l'adresse de la chaîne « Hello Word ».
 - L'opérateur + permet de concaténer des chaînes de caractères.
 - L'opérateur += permet d'ajouter une valeur à une variable en tenant compte de la précédente. Appliqué à un String, cela permet d'ajouter du contenu à la chaîne de caractère.
 - \n permet un saut de ligne.
- La méthode toString() de la classe Object retourne une représentation sous forme de String de l'objet.
 - On peut/doit la redéfinir dans chaque classe pour obtenir le résultat escompté par rapport aux données de la classe.
- L'instruction System.out.println(« Une chaîne de caractère ») permet d'afficher du contenu à l'écran :
 - System.out.println(« Une chaîne de caractère ») permet d'afficher « Une chaîne de caractère ».
 - Les instructions System.out.println(c1.toString()) et System.out.println(c1) et sont équivalentes : appel implicite de la méthode toString() dans la 2ème instruction.
 - De même, dans une méthode toString(), l'instruction return liste est équivalente à return liste.toString().

Interfaces Collection et List, classe ArrayList (Cf. Javadoc)

- Une (interface) Collection :
 - Est un objet qui contient plusieurs objets du même type (Ex : Torchon ou Serviette). Ce type peut être polymorphe (Ex : Linge de maison)
 - Peut être parcouru en séquentiel grâce à un Iterator [Cf. Atelier 2]. Une utilisation implicite (« transparente ») de l'itérateur est la suivante :


```
For (TypeElementsDeMaCollection refVersUnElement : maCollection) {
    // Manipulation de refVersUnElement ;
}
```
- Une (interface) List:
 - Est une Collection,
 - Est munie d'une relation d'ordre dans lequel la recherche du ième élément a un sens, mais qui peut également être parcouru en séquentiel,
 - Admet les doublons.
 - Un élément est ajouté dans la liste (add()), retiré (remove()), etc.
- Une (classe) ArrayList :
 - Est une List implémentée dans un tableau.

Abstraction et Encapsulation

- Une abstraction définit le comportement d'un objet (sa responsabilité) :
 - Que fait l'objet (fonctionnalités) et quelles données il manipule ?
 - Sans se soucier des détails d'implémentation (stockage données, algorithmes et langage pour traitement).
- L'encapsulation consiste à :
 - Isoler (mettre au même endroit) données (attributs) et comportement (méthodes) susceptibles d'être modifiés en cas d'évolution de l'application.
 - Contrôler autorisations accès par clients aux données et méthodes (public, private, etc.).

Atelier N°2

Atelier 2 - Objectifs

Compétences acquises à travers l'atelier (être capable de...)	Connaissances acquises à travers l'atelier	Étude préliminaire : vidéos à visionner avant l'atelier
• S'approprier l'architecture, la conception, le code d'une application dont le développement n'est que partiel. • S'adapter au mode de pensée d'un autre concepteur/développeur. • Identifier en quoi l'architecture et la conception proposées facilitent l'écriture, les tests et la future maintenance. • Comprendre la mise en œuvre de quelques Design Pattern (solutions à des problèmes de conception génériques) et découvrir pourquoi ils facilitent la maintenance, l'extension et la réutilisation de code.	• Piliers de la Programmation Orientée Objet : abstraction, encapsulation, héritage, polymorphisme, composition, délégation. • Interfaces Comparable et Comparator. • Interfaces Comparable, List et Iterator de Java. • Classes LinkedList et Collections de Java. • Classes anonymes • Mise en œuvre simple d'une architecture MVC (Model View Controller). • 1ère Découverte des Design Patterns : ▪ Proxy ▪ Template Method ▪ Composite ▪ Iterator	• Vidéo Youtube de Xavier Blanc « Programmation Orientée Objet - Cours 2 : Conception (Encapsulation et Responsabilités) » (10'). • Vidéo Youtube de Xavier Blanc « Programmation Orientée Objet - Cours 3 : Héritage » (18').

Atelier 2 - Déroulement

Objectifs et Moyens

Illustrer les notions d'héritage, de composition, de délégation, les avantages d'une architecture MVC et l'usage de Factory pour créer les objets.

Enrichir les classes « métier » du projet « fil-rouge » :

- La hiérarchie des Classes de Collections de cartes :
 - Deck, Hand, Board sont des collections de cartes qui ont des attributs et méthodes spécifiques et d'autres communs à factoriser dans AbstractCardsCollection.
 - On peut trier, mélanger, les éléments d'une Collection ou chercher le max dans la Collection.
- Les stratégies d'évaluation d'un pli avec différents algorithmes de comparaison des cartes.
- Les PlayerRender et CardRender, objets, sans setter, manipulés par les View.
- L'étude et les tests des View console et graphique et la composition de View (fournies).

1^{ère} activité : développement

- Le programme de test (TestAtelier2) est à dé-commenter au fur et à mesure des développements. Le résultat à obtenir est indiqué en commentaire à la fin de chaque ligne de code et en Annexe (fichier « Atelier 2 – Trace »).
- Les méthodes sont à compléter au fur et à mesure.
- Certaines méthodes sont à déplacer/supprimer dans/de certaines classes : pensez à les mettre en commentaire avant de les supprimer définitivement...
- Testez chaque ligne/bloc de test avant de passer à la suivante.
- Complétez le Carnet de Bord au fur et à mesure.

2^{ème} activité : correction du Carnet de Bord

Atelier 2 - Trucs et Astuces / Ce qu'il faut retenir

Classes Collections, interfaces Iterator et ListIterator (Cf. Javadoc)

- La classe **CollectionS** :
 - Propose plusieurs fonctions de manipulation des List : `sort()` (selon l'ordre naturel ou en fonction d'un `Comparator` – Cf. ci-dessous), `shuffle()`, `max()`, etc.
- Un (Interface) `Iterator` est un objet qui permet de parcourir une `Collection`. Un `ListIterator` peut parcourir des List bidirectionnelles.
 - Chaque `Collection` dispose d'une méthode qui permet de créer un itérateur (Ex : `Iterator it = maCollection.iterator()`).
 - L'utilisation implicite (« transparente ») d'un `Iterator` pour parcourir une `Collection` est la suivante :


```
For (TypeElementsDeMaCollection refVersUnElement : maCollection) {
    // Manipulation de refVersUnElement ;
}
```
 - L'utilisation explicite d'un `Iterator` pour parcourir une `Collection` est la suivante :


```
For (Iterator<TypeElementsDeMaCollection> it = maCollection.iterator() ; it.hasNext(); ) {
    refVersUnElement = it.next();
}
```
 - Avec un `ListIterator`, les instructions **successives** `it.next()` et `it.previous()` retournent le même élément (Cf. Javadoc).

Interfaces Comparable et Comparator (Cf. Javadoc)

- Implémenter l'interface `Comparable` revient à définir la manière de comparer selon l'ordre « naturel » 2 instances d'une classe dans la méthode `compareTo()`.
- Implémenter l'interface `Comparator` permet de définir d'autres comparateurs (selon d'autres algorithmes de comparaison) dans une méthode `compare()`.
- Les comparateurs sont principalement utilisés dans les algorithmes de tri, de recherche du min et max (`Collections.sort()`, `Collections.max()`) et les constructeurs de `Collections` (`TreeSet()`, `TreeMap()`).

Enum et values()

- La méthode `values()` est une méthode prédéfinie pour les enum qui renvoie un tableau contenant tous ses éléments dans l'ordre de leur déclaration :


```
enum JoursSemaine { LUNDI, MARDI, MERCREDI, JEUDI, VENDREDI, SAMEDI, DIMANCHE }
for (JoursSemaine jour : JoursSemaine.values()) { System.out.println(jour); }
```

Héritage

- Consiste à regrouper tous les éléments communs (attributs et méthodes) de plusieurs classes dans une classe mère :
 - Appelée aussi classe de base ou super-classe (Ex : AbstractCardsCollection).
 - Elle est souvent abstraite – déclarée « abstract » – (on ne peut pas l’instancier) car elle contient des méthodes abstraites dont elle est incapable de définir le code à son niveau.
 - Elle permet que le code commun ne soit pas dupliqué et d’éviter d’éventuelles incohérences de mise à jour.
- Une classe fille (ou classe dérivée) spécialise la classe mère :
 - Elle « hérite » de tous les attributs et méthodes de sa classe mère.
 - Elle donne un corps spécifique aux méthodes abstraites de la classe mère qui n’ont de sens qu’à cet étage de la hiérarchie.

Constructeurs multiples, Constructeur et Héritage

- Le constructeur d’une classe est généralement polymorphe, i.e. qu’on peut créer un objet de plusieurs façons, avec des paramètres de types différents.
- Si plusieurs constructeurs ont des arguments de même types, on écrit complètement celui qui les a tous et on écrit les suivants à partir de celui là. Ex :


```
Point (int x, int y, int z) { this.x = x ; this.y = y ; this.z = z ; }
Point (int x, int y) { this(x, y, 0) ; }
```
- Le constructeur d’une classe dérivée commence toujours par invoquer le constructeur de sa classe de base :


```
ColorPoint (int x, int y, int z, Color color) { super(x, y, z) ; this.color = color ; }
```

Composition et Délégation

- Un gros objet est souvent constitué de plusieurs plus petits objets qui ont chacun une responsabilité spécifique (Ex : 1 Voiture est composée d’un Moteur, de 4 Roue, etc.). Le gros objet, plutôt que de tout savoir faire, va déléguer à ses composants (enveloppés) le soin d’effectuer certaines actions.
- Il est également possible d’envelopper un objet dans un autre objet qui va soit restreindre, soit enrichir le comportement de l’objet enveloppé tout en s’appuyant sur ce qu’il sait déjà faire (sans le réécrire...). Ex : PlayerRender et CardRender. [Mécanisme très utilisé dans Design Patterns. Ex : Decorator, Adapter et Proxy]
- Cela facilite le développement/maintenance des classes (plusieurs petites au lieu d’une très grosse) et la réutilisation de code (un objet Roue peut être attribut d’une Voiture, d’une Moto, d’une Brouette, etc. et un objet PlayerRender délègue à son objet Player de faire tout ce qu’il sait faire) :

```
public final String getName() {
    return this.player.getName();
}
```

Polymorphisme et Conversion de type (casting / trans-typage)

- Est un mécanisme qui permet à un client de ne pas connaître, lorsqu'il invoque une méthode sur un objet, le type concret de l'objet (celui instancié avec un new). Il lui suffit de savoir que cet objet implémente la méthode. Une méthode de même nom peut donc exister dans des classes différentes et avoir des comportements différents.
- C'est la liaison dynamique (ou tardive) qui permet à Java de décider quelle méthode appeler à l'exécution en fonction du type réel de l'objet alors que la référence est de type polymorphe. Ex :

```
interface Animal { void parler( ) ;}
class Chien implements Animal { void parler( ) { System.out.println("Le chien aboie"); } }
class Chat implements Animal { void parler( ) { System.out.println("Le chat miaule"); } }
Animal monAnimal1 = new Chien();
Animal monAnimal2 = new Chat();
monAnimal1 parler (); // Le chien aboie
monAnimal2.parler (); // Le chat miaule
```

- Il existe plusieurs sortes de polymorphisme :
 - Polymorphisme d'interface : le client invoque des méthodes de même nom sur des objets déclarés comme instance d'une interface ; il n'y a pas nécessairement de hiérarchie d'héritage (Ex : IPlayer).
 - Polymorphisme d'héritage : des méthodes de même noms ont un comportement différent dans chaque classe dérivées (Ex : classes dérivées de AbstractGameEvaluator) : on parle de redéfinition.
 - Polymorphisme ad hoc : plusieurs méthodes de même nom dans la même classe avec des arguments de types différents ; on parle de surcharge.
- Conversion de type :
 - La conversion d'un objet déclaré pas son type polymorphe vers son type réel est parfois nécessaire : Card other = (Card) obj; [dans méthode equals()]
 - La conversion d'un objet vers son type polymorphe est implicite : Animal monAnimal1 = new Chien();

Composite [Design Pattern]

- Une structure de Composite permet aux clients de traiter une composition/collection d'objets et les objets individuels de la même manière. Ex. Nos IGameView sont vues par le Controller comme une seule View grâce à la classe GameViewables.

Proxy / Adapter / Decorator [Design Pattern]

- Un Adapter est un objet qui enveloppe un autre objet et fournit une interface différente pour y accéder.
- Un Proxy est un objet qui enveloppe un autre objet et en contrôle l'accès. Les 2 objets implémentent la même interface. Ex : CardRender et PlayerRender.
- Un Decorator est un objet qui enveloppe un autre objet pour fournir un nouveau comportement. Les 2 objets implémentent la même interface.

Template Method [Design Pattern]

- Une Template Method est une méthode qui définit dans une classe mère le squelette d'un algorithme et qui délègue une ou plusieurs parties de l'algo aux classes dérivées. Ex : Constructeur classe AbstractGameView et à de nombreux autres endroits dans le pgm.

Strategy [Design Pattern]

- Des comportements interchangeables sont encapsulés chacun dans une classe mais implémentent une interface commune. Le client utilise la délégation pour décider lequel utiliser. Ex : les implémentations de IGameEvaluator.
- Intérêts :
 - Un même comportement peut être utile à plusieurs clients (évite la redondance de code)
 - Le comportement peut être changé en cours d'exécution du pgm (un Humain a d'abord un comportement de Bébé, puis d'Enfant, d'Ado, d'Adulte mais c'est toujours la même personne (le même objet))

Atelier N°3

Atelier 3 - Objectifs

Compétences acquises à travers l'atelier (être capable de...)	Connaissances acquises à travers l'atelier	Étude préliminaire : vidéos à visionner avant l'atelier
• Enrichir une application Java en respectant les Principes de Responsabilité Unique (SRP), d'Ouverture-Fermeture (OCP) et de Substitution de Liskov (LSP). • Redéfinir des comportements existants lorsque c'est nécessaire (méthodes polymorphes). • Identifier les responsabilités de chaque interface/classe et répartir les données et traitements en conséquence, tout en garantissant un couplage faible et une forte cohésion. • Détecter des dysfonctionnements et les traiter. • Choisir l'organisation et le stockage des données appropriés en fonction des besoins et du contexte.	• Piliers de la Programmation Orientée Objet : toujours abstraction, encapsulation et d'avantage héritage, polymorphisme. • Framework de Collections en Java : interfaces Collection, List, Set, Queue, Map, et leurs implémentations. • Mise en œuvre simple d'une architecture MVC (Model View Controller). • 1^{ère} découverte du Design Pattern Strategy.	• Vidéo Youtube de Xavier Blanc « Programmation Orientée Objet - Cours 4 : IHM et MVC » (13'). Ne vous arrêtez pas sur les classes et méthodes du package swing, ce n'est qu'un exemple pour comprendre le principe.

Atelier 3 - Déroulement

Objectifs et Moyens

Achever les classes « Métier » et tester le projet Fil-Rouge dans son intégralité en connectant le Controller au Model et aux Views.

- Consolider les acquis avec le développement de la Hiérarchie des classes de Game pour tester les jeux de Bataille avec les différentes stratégies d'évaluation des plis.

1^{ère} activité : développement

- Prérequis : Dans la classe Card, réactivez la ligne « this.isFaceUp = false; » pour bien tester l'action de « retourner une carte ».
- Le programme de test est cette fois directement le Launcher de l'application (GameLauncher).
- Les instructions à commenter et dé-commenter sont celles du Controller (GameController) qui gère l'enchaînement des opérations.
- Les méthodes sont à compléter au fur et à mesure. Vous pouvez ajouter toute méthode private ou protected que vous jugerez nécessaire.
- Testez chaque ligne/bloc de test avant de passer à la suivante. Vous pouvez ajouter des instructions d'affichage ou de tests supplémentaires pour tester unitairement chacune de vos fonctions si vous le jugez nécessaire. Il existe déjà un booléen « trace » à réactiver dans GameController dans ce but.
- Complétez le Carnet de Bord au fur et à mesure.

2^{ème} activité : correction du Carnet de Bord

Atelier 3 - Trucs et astuces

Interface Map, classes TreeMap, HashMap (Cf. Javadoc)

- Une (interface) Map :
 - Une Map est une table associative qui, à la manière d'un dictionnaire, associe une clé et une valeur (par exemple mot/définition, titre/livre, etc.). La valeur peut être un simple élément ou une collection (Ex : une liste de synonymes associés à 1 mot)
 - La méthode `get(key)` de l'interface Map renvoie une référence vers la valeur à laquelle la clé spécifiée est associée ou null si cette Map ne contient aucune association pour la clé : Ex : `gamingMatMap.get(player)` renvoie une référence vers la carte appartenant au joueur.
 - La méthode `put(key, value)` de l'interface Map Associe la valeur spécifiée à la clé spécifiée. Si la Map contenait auparavant une association pour la clé, l'ancienne valeur est remplacée par la valeur spécifiée, sinon, le couple key/value est ajouté à la Map.
- Une Map n'implémente pas l'interface Iterable et ne peut donc pas être parcourue par un Iterateur (c'est relativement logique : on lit rarement un annuaire/dictionnaire du début à la fin...). Si néanmoins vous avez besoin d'accéder aux informations en séquentiel (comme c'est le cas dans notre pgm), parcourez le Set retourné par la méthode `entrySet()` :


```
Map<Player, Card> gamingMatMap ;
// ...
for (Entry<Player, Card> entry : gamingMatMap.entrySet()) {
    Player p = entry.getKey();
    Card c = entry.getValue();
    // ...
}
```
- Une Map est implémentée par :
 - Un TreeMap est une Map dont les éléments sont rangés en fonction de leur méthode `compareTo()` ou d'une méthode `compare()` (selon le constructeur utilisé)
 - Un HashMap est une Map dont les éléments sont rangés en fonction de leur hashCode.

Atelier N°4

Atelier 4 - Objectifs

Compétences acquises à travers l'atelier (être capable de...)	Connaissances acquises à travers l'atelier	Étude préliminaire : vidéos à visionner avant l'atelier
- Faire évoluer l'architecture d'une application Java tout en respectant la conception initiale et en garantissant la qualité du programme (respect principes S.O.L.I.D). - Concevoir les tests avant de coder. - Evaluer la conception et son impact sur la facilité de maintenance, d'extension et de réutilisation.	Consolider tous les acquis	

Atelier 4 - Déroulement

Pour les plus rapides :

- Développer de nouveaux types de jeu (Belote, etc.) sans remettre en question l'architecture et la conception existantes.
- Ou, développer une nouvelle View (graphique).
- Ou, permettre de jouer sur plusieurs écrans avec une communication des données à travers le net.

Conclusion / Déclulsion